Assessing the Ability of Open-Source LLMs to Identify Deceptive Clauses in Terms of Service Using Chain-of-Thought Prompting: A Comprehensive Study

Abdalla Alasmar

September 2, 2025

UNIVERSITY OF KENT

ACCESS TO A MASTER'S DEGREE OR POSTGRADUATE DIPLOMA DISSERTATION

In accordance with the Regulations, I hereby confirm that I shall permit general access to my dissertation at the discretion of the University Librarian. I agree that copies of my dissertation may be made for Libraries and research workers on the understanding that no publication in any form is made of the contents without my permission.

Notes for Candidates: by submitting your dissertation, you agree with the following:

1 Where the examiners consider the dissertation to be of distinction standard, one copy may be deposited in the University Library and/or uploaded into Moodle as an example of good dissertation for future students.

2 If a copy is sent to the Library, it becomes the property of the University Library. The copyright in its contents remains with the candidate. A duplicated sheet is pasted into the front of every thesis or dissertation deposited in the Library. The wording on the sheet is:

"I undertake not to use any matter contained in this thesis for publication in any form without the prior knowledge of the author."

Every reader of the dissertation must sign and date this sheet.

3 The University has the right to publish the title of the dissertation and the abstract and to authorise others to do so.

ABDALLA ALASMAR 02/09/2025

SIGNATURE DATE

FULL NAME ABDALLA SABIE ALASMAR

CERTIFICATE ON SUBMISSION OF DISSERTATION

I certify that:

1 I have read the University Degree Regulations under which this submission is made;

2 In so far as the dissertation involves any collaborative research, the extent of this collaboration has been clearly indicated; and that any material which has been previously presented and accepted for the award of an academic qualification at this University or elsewhere has also been clearly identified in the dissertation.

ABDALLA ALASMAR 02/09/2025

SIGNATURE DATE

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Budi Arief, for their invaluable guidance, continuous support, and constructive feedback throughout this research project.

I would also like to thank the University of Kent and the School of Computing for providing the academic environment for this study.

Abstract

Terms of Service (TOS) agreements shape our interactions with digital services, defining how our data is collected, shared, and retained. They specify our rights and responsibilities, often concealing clauses that could undermine user privacy or impose burdensome conditions. Many users rarely read these agreements in full, and the rise of "dark patterns" means that troubling clauses are frequently obscured by complicated language. On the other hand, large language models (LLMs) have emerged as powerful tools for analyzing text. This dissertation explores whether open-source LLMs,specifically Mistral-7B-Instruct, Meta-Llama-3-8B-Instruct, and Microsoft Phi-2, can effectively identify deceptive or privacy-violating clauses within TOS documents. Through a binary classification framework (deceptive versus legitimate), I evaluate whether Chain-of-Thought (CoT) prompting enhances performance compared to straightforward classification prompts. The study begins with a review of relevant literature on TOS deception, dark patterns, legal technology, and prompting strategies for LLMs. A reproducible experimental methodology is detailed, featuring a balanced dataset of ten legitimate and ten deceptive clauses. The implementation is described through a comprehensive analysis of the accompanying notebook code, covering dataset construction, prompt generation, response parsing, evaluation metrics, visualization, and qualitative error analysis.The study's results show varied performance across the models, with Chain-of-Thought (CoT) prompting significantly improving the performance of Mistral-7B-Instruct. Eventually, this study argues that CoT prompting has the potential to enhance the transparency and performance of deception detection, though its effectiveness varies by model.

Contents

1 Introduction **3**

1.1 Research Contributions 4

1.2 Why Terms of Service Analysis Matters 4

1.3 Large Language Models and Chain-of-Thought: Theoretical Foundations . 5

1.4 Dissertation Structure 5

2 Literature Review **7**

2.1 Deceptive Practices in Terms of Service Agreements 7

2.2 Computational Approaches to Legal Document Analysis 7

2.3 Application of Large Language Models in Legal Analysis 8

2.4 Advances in Chain-of-Thought Prompting 8

2.5 Previous Research 9

2.6 Comparison with Prior Work on Unfair Term Detection 10

2.7 Technical Foundations 11

2.8 Methodologies for LLM Performance Enhancement 11

2.9 Strategic Deception and Alignment in Language Models 12

3 Research Problem and Objectives **13**

3.1 The Core Question 13

3.2 Objectives 13

3.3 Expected Outcomes 14

4 Methodology **15**

4.1 Hardware and Software Specifications 15

4.2 Experimental Environment and Setup 15

4.3 Building the Dataset 16

4.4 Model Loading and Configuration 16

4.5 Prompt Generation 17

4.6 Clause Evaluation and Response Parsing 17

4.7 Experiment Execution 18

4.8 Performance Evaluation 18

4.9 Visualizations and Qualitative Analysis 18

4.10 Error Analysis 19

4.11 Analysis of Response Verbosity 19

4.12 Methodological Constraints 19

5 Results **20**

5.1 Meta-Llama-3-8B 20

5.2 Mistral-7B-Instruct 22

5.3 Microsoft Phi-2 23

5.4 Comparing Across Models 25

5.5 Response Length and Qualitative Quality 26

6 Discussion **27**

6.1 Interpreting Model Performance 27

6.2 Qualitative Insights from Chain-of-Thought Responses 27

6.3 Error Patterns and Ethical Considerations 28

6.4 Implications for Legal Tech and Policy 29

6.5 Future Directions 29

7 Conclusion **30**

7.1 Assessing Open-Source LLM Performance 30

7.2 Evaluating Chain-of-Thought Impact 30

7.3 Creating a Reproducible Benchmark 31

7.4 Analyzing Performance Differences 31

7.5 Unexpected Findings 31

7.6 Implications for the Research Domain 32

7.7 Future Directions 32

7.8 Concluding Remarks 33

A Complete Experiment Code **34**

8 References **50**

Chapter 1

Introduction

In our current digital landscape, Interactions frequently take place through multiple platforms. From social media to online banking, people routinely accept Terms of Service agreements that can be lengthy and complex. These documents grant substantial rights to service providers, including permissions to collect user data indefinitely, share it with unknown parties, modify terms without notice, and require binding arbitration that waives traditional judicial recourse [2]. Research in human-computer interaction reveals that most users do not thoroughly read TOS documents, and the complicated nature of these agreements discourages careful inspection [3]. Adding to the issue, dark patterns, which are strategies designed to manipulate user behavior, are common in both user interfaces and contractual language. Examples include vague clauses regarding data sharing, pseudo-anonymous data usage, and indefinite retention policies. Given that users rarely examine these clauses, Automated tools that can identify deception or privacy violations within TOS are necessary. Large language models have emerged as powerful engines for understanding and reasoning about natural language. Models such as GPT-4, Llama-3, Mistral-7B, and Phi-2 are trained on vast collections and are capable of tasks like summarization, translation, question-answering, and classification [4]. However, these models exhibit varied performance based on architecture, parameter count, training data, and prompting strategies. Chain-of-thought prompting has shown promise in enhancing reasoning by guiding the model to think through problems step by step rather than providing immediate answers. In tasks that require arithmetic reasoning, logic puzzles, or theorem proving, CoT prompts encourage models to express intermediate steps, often leading to more accurate conclusions. however, whether this prompting strategy can help in detecting deceptive contractual language remains an open question [15]. Contracts involve domain knowledge, subtle linguistic cues, and legal nuances; thus, a model's ability to navigate these complexities may benefit from a structured, step-by-step approach. This dissertation focuses on the research question described in the proposal: Can open-source large language models, such as Mistral-7B or LLaMA-3-8B, effectively detect deceptive or privacy-violating clauses in Terms of Service documents using chain-of-thought prompting? I examine this issue from multiple disciplinary perspectives, combining insights from

legal studies, computational linguistics, machine learning, and digital ethics. The following sections summarize the state of the art, present a detailed experimental methodology based on the provided notebook code, analyse preliminary results, and discuss the implications and limitations of this work. This research aims to establish an experiment that could help in future studies on AI-assisted legal document analysis while highlighting both the potential and the challenges involved in applying LLMs in high-stakes domains.

1.1 Research Contributions

Empirical evidence on prompt engineering effectiveness: We provide the first systematic comparison of Chain-of-Thought prompting versus standard classification prompts for detecting deceptive clauses in Terms of Service documents, revealing model-dependent effects that challenge assumptions about universal prompt improvements. Open-source model benchmarking: We establish baseline performance metrics for three widely-used open-source LLMs (Mistral-7B-Instruct, Meta-Llama-3-8B-Instruct, and Microsoft Phi-2) on legal deception detection, demonstrating that Mistral-7B shows a 57% relative improvement with CoT prompting whilst Phi-2 experiences performance degradation. Reproducible evaluation framework: We provide a publicly available dataset of 20 annotated TOS clauses and accompanying implementation code, enabling future researchers to replicate and extend our findings in legal NLP applications.

1.2 Why Terms of Service Analysis Matters

Terms-of-Service contracts form the legal backbone of digital interactions. They inform users about how their data will be utilised, what rights they retain or surrender, and the solutions available in the event of disagreements [7]. Ideally, TOS should promote transparency and trust; however, they often obscure critical details behind complex legal idiom. Many agreements contain clauses that allow the service to share personal data with branches, track behavioral data, retain information indefinitely[1], or impose binding arbitration. Such provisions can undermine privacy and independence, leaving users with little recourse to negotiate better terms. Scholars have pointed out that the complexity of TOS documents undermines the principle of informed consent. The existence of deceptive clauses can lead to privacy violations, data breaches, or exploitation of personal data [3]. Automated analysis tools could act as early warning systems, highlighting potentially harmful clauses for further examination. Previously, rule-based systems have been employed to identify patterns in legal documents, but these methods require extensive domain expertise and often fail to generalize effectively [8]. In contrast, machine learning approaches, particularly those that utilise natural language processing (NLP), provide greater flexibility. Pre-trained models can be fine-tuned or prompted to classify text segments as either deceptive or legitimate. Nevertheless, the high stakes associ-

ated with legal interpretation requires a rigorous evaluation of any automated approach. Misclassifications could result in missing significant privacy violations or generating false alarms, thus undermining the trust in the system.

1.3 Large Language Models and Chain-of-Thought: Theoretical Foundations

Large language models are neural networks trained on extensive datasets to predict the next token in a sequence. Their capability to produce coherent and contextually appropriate text derive from patterns learned across billions of examples[13]. Open-source models like Meta-Llama-3-8B and Mistral-7B are accessible alternatives to non-open-source models such as GPT-4, allowing for transparency and reproducibility in research [14]. The open-source nature of these models allows researchers to audit their training data, modify architectures, and tune performance for specialized tasks. However, their smaller parameter counts compared to non-open-source models may limit their ability to capture the subtleties of contractual language. Evaluating open-source models on real tasks will provide insight into their viability for legal analysis. Chain-of-thought prompting instructs the model to reason step by step rather than generating immediate answers. A typical CoT prompt might state, 'Let's think step by step', before asking the question, prompting the model to express its reasoning. Research has demonstrated that CoT prompts can enhance performance on arithmetic, commonsense reasoning, and multi-step tasks [10]. The underlying hypothesis assumes that prompting models to express their reasoning encourages them to consider multiple facets of the problem, which may reduce the probability of hallucination. However, CoT often leads to longer responses and may introduce unnecessary detail. In high-stakes contexts, ensuring that reasoning is accurate is crucial, as verbosity without substance can be detrimental. This dissertation seeks to ascertain whether CoT prompting enhances classification accuracy and interpretability for TOS deception detection.

1.4 Dissertation Structure

This dissertation is organized as follows: Section 2 reviews related work on deceptive clauses, dark patterns, and computational approaches to legal document analysis. It also explores prior studies on chain-of-thought prompting and its applications. Section 3 presents the research problem and objectives as specified in the proposal, including the research question, objectives, and expected contributions. Section 4 details the methodology, describing dataset creation, model selection, prompt design, evaluation procedures, and the implementation of the experiment in the notebook. Each component of the code is explained, and limitations are discussed. Section 5 presents the experimental results; The results section presents a detailed analysis of the experimental outcomes. Section

6 critically discusses the findings, focusing on CoT effects, misclassifications patterns, and ethical considerations. Finally, Section 7 concludes the dissertation by summarizing contributions and outlining future research directions.

Chapter 2

Literature Review

2.1 Deceptive Practices in Terms of Service Agreements

"Dark patterns" refer to design strategies employed in user interfaces and textual content that manipulate user decisions. Within the context of TOS, dark patterns often exhibit as clauses that obscure data sharing, limit user rights, or impose unfavorable arbitration procedures [2]. Researchers have cataloged various categories of deceptive clauses, including rights-capture legislations that allow companies unlimited licenses to use user content, vague sharing clauses permitting information sharing with undefined partners, and pseudo-anonymous clauses that claim data is anonymized while still allowing reidentification. Other examples include fake deletion clauses that promise account deletion while retaining data indefinitely and silent update clauses that permit term modifications without user notification. These patterns exploit the power imbalance between corporations and users, capitalizing on the tendency of users to accept terms without reading them. Legal scholars argue that deceptive clauses undermine informed consent and may violate consumer protection laws [3]. For example, clauses that require arbitration and waive class actions could limit the ability of users to seek compensation for collective harms. Unclear authority clauses may allow data transfer to countries with weak privacy protections[1]. Academics and regulators have called for improved transparency and automated tools to assist consumers and advocates in identifying problematic language. The task of TOS deception detection aligns with this objective by attempting to automatically detect clauses that warrant a closer examination.

2.2 Computational Approaches to Legal Document Analysis

Natural language processing has long been applied to legal documents. Early efforts focused on rule-based systems and keyword matching [8]. For example, researchers have utilized dictionaries of privacy-related terms to identify clauses discussing data usage. However, rule-based systems struggle to manage the complexity and variability of natural language. Machine-learning approaches, particularly those employing supervised

learning, have produced greater success. Support vector machines, random forests, and neural networks have been deployed to classify clauses into categories such as "data collection," "third-party sharing," and "user rights." More recently, deep learning models like BERT, GPT-2, and domain-specific LLMs have achieved state-of-the-art performance in contract analysis. These models can capture contextual relationships and subtle variations in wording. Despite these advancements, challenges persist. Legal language can be highly technical[17], and minor changes in wording can alter a clause's meaning. Models may also latch onto superficial cues rather than effectively understanding the legal implications. Additionally, many published datasets used for contract analysis are small or proprietary, limiting reproducibility. Creating balanced, publicly available datasets is essential for fair evaluation. The dataset proposed in this study aims to address this need by providing ten legitimate and ten deceptive clauses drawn from known examples and manually designed dark patterns. Though small, this dataset can serve as an initial benchmark for examining whether open-source models can generalize to TOS clauses.

2.3 Application of Large Language Models in Legal Analysis

Large language models have demonstrated remarkable capabilities in summarizing legal documents, answering bar exam questions, and drafting contracts. For instance, GPT-3 and GPT-4 can generate reasonable legal analyses when given well-structured prompts [7]. However, reliability is not guaranteed, and concerns about hallucination and misinterpretation remain. In high-impact domains like legal advice, even minor errors can have significant consequences. The benefits of open-source models include transparency and cost-effectiveness, but their performance may lag behind non-open-source models due to fewer parameters and differences in training data. Evaluating models such as Mistral-7B, Meta-Llama-3-8B, and Phi-2 in legal tasks helps determine whether they can provide meaningful insights or if further fine-tuning is necessary. Mistral-7B is a 7-billion-parameter model trained on a mix of code, literature, and web text. It has garnered praise for its efficiency and strong performance relative to its size. Meta-Llama-3-8B, part of Meta's Llama-3 family, consists of 8 billion parameters and emphasizes instruction tuning and safety. Phi-2 is a smaller model focused on efficiency and robust performance across languages. These models are accessible via the Hugging Face hub, making them appealing for open research. Understanding how they handle complex classification tasks like TOS deception detection, where legal and ethical nuances intersect, is crucial for assessing their broader suitability for legal analytics.

2.4 Advances in Chain-of-Thought Prompting

Chain-of-thought prompting gained attention following research[16] indicating that directing models to think step by step can improve reasoning on tasks that involve multiple

steps or internal discussions. In arithmetic and logic problems, CoT prompts lead to more explicit reasoning and higher accuracy [10]. The underlying mechanism appears to be "implicit structure," where the model utilises the natural language reasoning it generates to guide the answer. However, the effects of CoT can differ depending on the task and model. In certain cases, CoT prompts produce longer responses without improving final accuracy; in others, they may lead models to hallucinate reasonable but incorrect reasoning. This technique proves particularly promising for tasks requiring the evaluation of multiple factors, as is the case in TOS deception detection. The proposal under consideration explicitly contrasts two types of prompts: a straightforward classification prompt asking whether a clause contains deception or a privacy violation, and a chain-of-thought prompt instructing the model to reason through several specific aspects, including vague language about data usage, excessive rights, restrictions on user rights, unclear consent, and data retention policies. By analyzing how models respond to each prompt type, I can assess whether CoT enhances reasoning and classification accuracy. Additionally, the lengthier responses generated under CoT can be qualitatively examined to determine if they offer clearer justifications and greater transparency, although at the cost of verbosity and potential hallucination.

2.5 Previous Research

The growing interest in detecting unfair contractual clauses has motivated several recent studies that complement this work.[2] compiled a dataset of twelve questions covering common unfair terms in privacy policies and used it to benchmark both open-source and commercial models. They reported that open-source models occasionally outperformed private systems, but overall performance was only slightly better than random guessing. The best results were achieved by the ChatGPT-4 model, underscoring the complexity of the task and the need for richer datasets. Nayak [3] introduced AutoBot, a two-stage pipeline that analyzes website screenshots and textual features using a custom language model to detect deceptive patterns in real time. Implemented as a local Chrome extension, AutoBot emphasizes privacy preservation and aims to enable users and regulators by identifying deceptive patterns during browsing. Similarly, Boujarwah[4] developed TermMiner, a large-scale pipeline that crawled 8,251 shopping websites to extract 1.8 million TOS clauses and build the ShopTC-100K dataset. They suggested a taxonomy of 22 types of unfavorable financial terms and introduced TermLens, an LLM-based detector that achieved an F1-score of 94.6% on annotated data while maintaining a low false positive rate. This study found that 42% of the top 100,000 shopping sites contained at least one unfavorable term, highlighting the prevalence of problematic clauses.

2.6 Comparison with Prior Work on Unfair Term Detection

Mirgita Frasheri 's paper[2], "Are LLM-based methods good enough for detecting unfair terms of service?" presents a focused evaluation of LLMs on the task of identifying unfair terms. The authors constructed a dataset of twelve questions derived from privacy policies and consumer contracts, asking models to determine whether the clauses were unfair. They tested eleven models spanning open-source and commercial LLMs, producing sobering findings: performance was only marginally better than random guessing, and while some open-source models occasionally outperformed commercial ones, the best results came from GPT-4. The authors concluded that current LLMs are not yet reliable enough for detecting unfair terms and that better datasets and domain-specific training are necessary. This dissertation differs from Mirgita Frasheri's study[2] in several key areas. First, the research questions and datasets differ. While Frasheri's [2] study focuses on unfairness in privacy policies, often involving questions about whether clauses violate legal norms or consumer rights, this work targets deceptive clauses and dark patterns in general TOS documents. Our dataset comprises twenty hand-curated clauses (ten legitimate and ten deceptive), drawing on patterns such as rights-grab clauses, vague sharing, and pseudo-anonymization. Though both datasets are small, ours is balanced and explicitly designed to test the detection of deception rather than fairness. Second, the experimental methodology differs.Frasheri [2] employed a multiple-choice format and compared models purely on their final answers, without exploring prompt engineering beyond basic question formatting. In contrast, this study evaluates the impact of chain-of-thought prompting. Each clause is presented under two prompt strategies: a simple classification instruction and a step-by-step reasoning prompt that prompts the model to consider vagueness, rights grabs, consent, and retention policies before deciding. By comparing these prompts, we can assess whether explicit reasoning improves classification accuracy and interpret ability. Third, the model selection differs. Frasheri's study [2] evaluated a mix of non-open-source and open-source models, concluding that GPT-4 performed best. This research focuses on accessible open-source models, Mistral-7B-Instruct, Meta-Llama-3-8B, and Phi-2, to determine whether community-maintained models can deliver acceptable performance. we measure accuracy, precision, recall, and F1-score and analyse confusion matrices. The results indicate heterogeneous effects of CoT: significant improvements for Mistral-7B but marginal or negative effects for Llama-3-8B and Phi-2. These findings suggest that open-source models can benefit from prompt engineering, whereas Frasheri's study [2] emphasized the superiority of non-open-source models and the need for more training data.

2.7 Technical Foundations

The effectiveness of any deception detection approach is influenced by the underlying model architecture. The modern era of language modeling was enabled by the Transformer architecture introduced by Vaswani in 2017. Their seminal paper, Attention Is All You Need, proposed a sequence-to-sequence model based alone on multi-head attention mechanisms and eliminated the need for recurrence and convolutions. The Transformer achieved state-of-the-art results on English–German and English–French translation tasks while offering superior parallelism and efficiency [11]. This architecture forms the backbone of most contemporary LLMs. Building on this foundation, Shao [14] published a comprehensive survey of large language models that charts the evolution from statistical language models to pretrained transformer models, discusses the scaling effects that give rise to emergent abilities, and categorizes research into four major aspects: pretraining, adaptation tuning, utilization, and capacity evaluation. The survey notes that once parameter counts cross certain thresholds, models exhibit new capabilities not present in smaller models, and both industry and academia have been pushing LLM research forward. Concurrently, legal scholars have examined the suitability of LLMs for legal reasoning. Wang[7] evaluated open- and closed-source models on English and Chinese legal tasks, finding that while LLMs excel at summarizing legal texts, they often struggle with nuanced legal reasoning and may misinterpret statutory provisions. These findings underscore the importance of careful prompt design and highlight the potential limitations of applying general-purpose LLMs in high-impact domains like law.

2.8 Methodologies for LLM Performance Enhancement

Beyond general architecture, LLM performance can be significantly altered through fine-tuning. Parthasarathy[10] presented The Ultimate Guide to Fine-Tuning LLMs, a comprehensive review that traces the historical evolution of fine-tuning methodologies and synthesizes best practices. They propose a structured seven-stage pipeline, from data preparation and model initialization to hyperparameter tuning, deployment, monitoring and discuss parameter-efficient techniques such as Low-Rank Adaptation (LoRA) and half fine-tuning that reduce computational costs while maintaining performance. Advanced strategies like Mixture of Experts, Mixture of Agents, Proximal Policy Optimization, and Direct Preference Optimization are explored alongside optimizations for inference on distributed or cloud platforms. This report emphasizes emerging challenges related to multimodal fine-tuning, scalability, privacy, and accountability, offering actionable guidance for practitioners. Applied work has shown that open-source models can be effectively adapted to legal contexts. In a case study on the Indian constitution, Gupta[8] utilised Hugging Face embeddings and a Flan-T5 model within the LangChain framework to summarize constitutional provisions and extract key information. They found

that open-source models produced high-quality summaries and saved researchers significant time compared to manual analysis. These results suggest that, when appropriately fine-tuned or prompted, smaller models can deliver meaningful legal insights. This study explores this possibility by evaluating fine-tuned open-source models on TOS clauses and experimenting with prompt engineering rather than additional training.

2.9 Strategic Deception and Alignment in Language Models

Chain-of-thought prompting raises important alignment concerns. While it can improve transparency and reasoning, explicit thought traces also expose the risk of strategic deception. Wang [15]investigated this phenomenon in When Thinking LLMs Lie, where they induced and detected strategic deception in CoT-enabled models using representation engineering. They derived "deception vectors" via linear artificial tomography, achieving 89% detection accuracy and demonstrating a 40% success rate in extracting context-appropriate lies without explicit prompts. The authors argue that honesty and alignment remain pressing challenges for models that articulate their reasoning, proposing activation-steering techniques to mitigate deceptive behaviors. These findings are relevant to this work as they caution against the potential for CoT outputs to be manipulated or misinterpreted, highlighting the importance of evaluating both the correctness and authenticity of model reasoning. Incorporating insights from alignment research ensures that the pursuit of transparency does not inadvertently create new avenues for misuse.

Chapter 3

Research Problem and Objectives

3.1 The Core Question

The core research question driving this study is: "Can open-source large language models (LLMs), such as Mistral-7B or LLaMA-3-8B, effectively detect deceptive or privacy-violating clauses in Terms of Service (TOS) documents using Chain-of-Thought (CoT) prompting?" This question arises from the intersection of digital ethics, user privacy, and artificial intelligence. It recognizes the potential of LLMs to parse complex language, while acknowledging that legal documents present unique challenges that may require structured reasoning prompts.

3.2 Objectives

Four main objectives guided this research: 1. To assess the performance of small, open-source LLMs in identifying deceptive or unethical contractual language. Focusing on open-source models promotes reproducibility and accessibility. Performance will be measured through accuracy, F1-score, and other metrics derived from the classification of 20 TOS clauses into "deceptive" or "legitimate." 2. To evaluate the impact of Chain-of-Thought prompting on the accuracy and interpretability of model responses. This involves comparing the results of simple classification prompts with CoT prompts that encourage step-by-step analysis of clauses. Interpretability will be assessed by analyzing the reasoning contained in model outputs, identifying whether CoT responses provide clearer explanations or reveal hallucinations. 3. To construct a lightweight, publicly reproducible benchmark for deception detection in TOS. The benchmark comprises a balanced dataset of 20 clauses, evenly split between legitimate and deceptive patterns. The dataset will be released along with the prompts and classification results to allow other researchers to replicate and extend the experiment. 4. To analyse and visualize the differences between CoT and non-CoT performance, both quantitatively and qualitatively. Quantitative analysis will include metrics like accuracy, precision, recall, F1-score, and the number of unclear predictions. Qualitative analysis will examine example re-

sponses where CoT improved classification or introduced errors, in addition to response length distribution and error types.

3.3 Expected Outcomes

I anticipate that chain-of-thought prompting will lead to improved accuracy, increasing performance from approximately 60% with a simple prompt to around 80% under CoT. I also predict improved justifications and clearer explanations under CoT, particularly in ambiguous cases, while noting the risk of hallucinated reasoning. Deliverables will include a written research report, the source dataset, prompts, and an analytical notebook with visualizations such as bar charts and error examples. The significance of this work lies in demonstrating whether accessible, open-source models can assist in flagging potentially deceptive clauses and whether CoT prompting offers tangible benefits. Even if performance remains modest, the work will establish a benchmark and highlight areas requiring further research.

Chapter 4

Methodology

This section explains the methodology used to implement and execute the experiment. This section demonstrates the methodology implemented in the experiment notebook, linking each component to the research objectives. The code provided in the notebook serves as a reference implementation and guiding you through its key functions, classes, and design choices.

4.1 Hardware and Software Specifications

Table 4.1: Hardware Specifications

Component	**Specification**
GPU	NVIDIA RTX 3060Ti 8GB
CPU	Intel Core i9-10850K
RAM	48GB DDR4
Storage	1TB NVMe SSD
Operating System	Windows11 Pro 64-bit/24H2
Python Version	3.10
CUDA Version	12.1
PyTorch Version	2.0.1

4.2 Experimental Environment and Setup

The experiment is implemented in a Jupyter notebook. The notebook begins with installation commands for required libraries, including transformers, bitsandbytes, torch, datasets, scikit-learn, pandas, matplotlib, seaborn, and tqdm. It also logs into the Hugging Face Hub to access pretrained models. The environment is configured to utilise GPU if available, with a fallback to CPU otherwise. By using bitsandbytes, models can be loaded in 4-bit precision, which reduces memory usage and facilitates running large models on consumer hardware .

4.3 Building the Dataset

The dataset is constructed through a Python class TOSDataset defined in the notebook. Upon initialization, the class calls its "create-dataset" method to produce a list of dictionaries representing clauses. Each dictionary contains the text of the clause, a label ("legitimate" or "deceptive"), and metadata indicating the source or pattern. Legitimate clauses are sourced from privacy-respecting services and emphasize transparency, user rights, and consent. For example, one legitimate clause states: "We will never sell your personal data to third parties," labeled as "legitimate" with the source noted as "privacy-focused." Another clause reads: "You can delete your account and all associated data at any time through your account settings," also labeled legitimate. Additional legitimate clauses affirm encryption of data, explicit consent for sharing, retention limits, and timely breach notifications. In contrast, deceptive clauses embody dark patterns and privacy violations. These include rights-grab clauses granting the company unlimited licenses to use user content, vague sharing statements that permit data sharing with affiliates without clear consent, and fake deletion clauses that retain data indefinitely while claiming to process deletion requests. Other deceptive clauses reserve the right to modify terms without notice, assert that anonymized data may be used for research (with the risk of reidentification), and require users to accept promotional communications by default. These examples collectively represent common types of deception in TOS and provide a simple yet diverse training set. The dataset consists of ten legitimate and ten deceptive clauses, ensuring balance between classes. After creation, the clauses are converted into a Pandas DataFrame for easier manipulation and analysis. The notebook prints the total number of clauses, the distribution of labels, and sample entries to verify correct construction. Although the dataset size is limited, it serves as a foundational step toward building larger benchmarks and allows the experiment to focus on prompt design and model evaluation.

4.4 Model Loading and Configuration

The notebook defines a class LLMEvaluator to include model loading, prompt generation, clause evaluation, and response parsing. Upon initialization, the evaluator accepts a model-name and an optional cache directory. If a cache directory is specified, it is created and used to store model weights, enabling offline inference. The evaluator prints diagnostic messages about the model name and cache directory for transparency. It utilises Hugging Face's AutoTokenizer and AutoModelForCausalLM classes to load the tokenizer and model. If the tokenizer lacks a padding token, the evaluator sets it to the end-of-sequence token. The model is loaded with 4-bit precision (load-in-4bit=True) and uses torch-dtype=torch.float16 when a GPU is accessible, falling back to full precision otherwise. Device mapping is set to "auto" for GPU or None for CPU, and low-cpu-mem-

usage=True helps to reduce memory footprint. After loading, if no GPU is available, the model is explicitly moved to the CPU, and a success message is printed.

4.5 Prompt Generation

The evaluator includes a method called generate-prompt that constructs prompts based on whether CoT is activated. For the no-CoT condition, the prompt simply states: "Determine if the following Terms of Service clause contains deception or privacy violations. Clause: [clause] Answer (Deceptive/Legitimate):". This prompt is straightforward and instructs the model to provide a binary classification. In the CoT condition, the prompt is significantly more detailed, guiding the model to analyse the clause step by step while explicitly considering five aspects: (1) vague or ambiguous language about data usage, (2) excessive rights granted to the company, (3) limitations on user rights or legal recourse, (4) data sharing without clear consent, and (5) unclear data retention policies. The prompt ends with the phrase "Analysis: Let me think through this step by step," which further signals the model to externalize its reasoning. This design aligns with CoT prompting literature and aims to structure the model's reasoning process.

4.6 Clause Evaluation and Response Parsing

The core evaluation logic resides in the evaluate-clause method. Given a clause and a boolean use-cot, the method generates the appropriate prompt and tokenizes it using the model's tokenizer. It then feeds the inputs through the model with torch.no-grad() to disable gradient computation, using the generate method with sampling (temperature=0.7, top-p=0.95) to produce a response. Temperature and top-p sampling encourage diversity in responses while reducing extreme randomness. The generated sequence is decoded and trimmed of the original prompt to isolate the model's answer. The evaluator then calls parse-response to classify the answer. The parse-response method employs simple keyword matching to determine whether the model's output indicates a deceptive or legitimate classification. It converts the response to lowercase and counts occurrences of deceptive keywords (e.g., "deceptive", "misleading", "violation", "yes") and legitimate keywords (e.g., "legitimate", "fair", "no"). If the deceptive score exceeds the legitimate score, the clause is classified as deceptive; conversely, if the legitimate score is higher, it is classified as legitimate. In cases where scores are tied, further checks look for specific phrases like "not deceptive" to assign a label; otherwise, the classification is set to "unclear." This heuristic avoids the need for a separate classifier but may misinterpret complex responses, highlighting a limitation of the current approach. Future work could involve training a dedicated classifier for more reliable interpretation of model outputs.

4.7 Experiment Execution

To run the full experiment across the dataset, the notebook defines the ExperimentRunner class. It accepts an evaluator instance and the dataset DataFrame. The run-experiment method iterates through each clause in the dataset and evaluates it twice: once with use-cot=False and once with use-cot=True. The results are appended to a list as dictionaries containing the clause text, true label, predictions, and raw responses for both conditions. Intermediate results are saved to JSON files every five clauses to prevent data loss during long-running experiments. After processing all clauses, the method returns a DataFrame of results. This design ensures that results for no-CoT and CoT prompts are directly comparable and recorded alongside each clause.

4.8 Performance Evaluation

Once predictions are obtained, the notebook uses the Performanceanalyser class to compute metrics. The calculate-metrics method filters out "unclear" predictions, as they cannot be unambiguously mapped to labels. It calculates accuracy, precision, recall, and F1-score for both no-CoT and CoT conditions using scikit-learn metrics functions . The method also counts the number of unclear predictions. It returns a dictionary with metrics, and the print-metrics method formats and displays the results. Additionally, it computes the absolute and percentage improvement from no-CoT to CoT for each metric when both sets are available. These metrics provide quantitative evidence of whether CoT prompting leads to better classification performance.

4.9 Visualizations and Qualitative Analysis

The notebook includes a function called create-visualizations that generates six plots summarizing results: (1) accuracy comparison, (2) F1-score comparison, (3) precision versus recall, (4) no-CoT confusion matrix, (5) CoT confusion matrix, and (6) performance improvement. Bars and heatmaps are annotated with values and labeled axes. The figures are saved as PNG files for inclusion in reports. These visualizations help interpret metrics at a glance and identify patterns, such as whether one condition produces more false positives or false negatives. To examine cases where CoT improved classification, the function analyse-cot-improvements filters the results DataFrame to find instances where the CoT prediction matches the true label while the no-CoT prediction is incorrect. It outputs to three examples, showing the clause, true label, both predictions, and the first 200–400 characters of each response. This qualitative analysis reveals how CoT reasoning may alter the model's decision and whether the reasoning appears sensible.

4.10 Error Analysis

The error-analysis function identifies and counts misclassifications. It separates errors into false positives (legitimate clauses classified as deceptive) and false negatives (deceptive clauses classified as legitimate) for both no-CoT and CoT conditions. By analyzing error types, we can determine whether the model tends to be overly cautious (resulting in more false positives) or too permissive (leading to more false negatives).

4.11 Analysis of Response Verbosity

The notebook ends with an analysis of response length. The response-analysis function computes the length of each response (in terms of character count) and prints the mean and median lengths for both no-CoT and CoT conditions. It plots histograms showing the distribution of response lengths and marks the mean with a dashed line. Given that longer responses are expected under CoT, understanding the increase in length helps assess whether the additional verbosity is manageable or burdensome. Finally, the export-results function saves the results DataFrame, metrics, and dataset as CSV files and exports a JSON report summarizing experiment metadata and performance. The final printout summarizes dataset size, model information, key findings (accuracies, F1-scores, and improvements), and observations about CoT responses. This summary is intended for quick reference and emphasizes the most noticeable findings.

4.12 Methodological Constraints

While comprehensive, this methodology has its limitations. The dataset of 20 clauses is too small to adequately train or robustly evaluate machine-learning models. The experiment relies entirely on the models' pretraining and prompt effectiveness, rather than learning from data. Additionally, the classification of model outputs employs simple keyword heuristics; more sophisticated parsing may produce different results. The selection of legitimate and deceptive clauses might not cover the full diversity of TOS language, as some clauses may be ambiguous even for human evaluators. Furthermore, models are run with generative sampling, meaning that responses can vary from one run to another; results may need to be averaged over multiple seeds for stability. Lastly, the open-source models evaluated here may have limitations in understanding legal context compared to larger non-open-source models.

Chapter 5

Results

In this section, I present the empirical results for the three evaluated models. For each model, I compare the performance of the simple classification prompt (no-CoT) with that of the chain-of-thought prompt (CoT). The reported metrics include accuracy, precision, recall, F1-score, the number of unclear predictions, and the mean response length. All metrics exclude cases where the evaluator could not determine a clear label (the unclear count is reported separately) and correspond to the evaluation procedure described in Section 4.7. The accompanying figures summarize accuracy, precision, recall, and F1-scores and provide confusion matrices for both prompt types.

5.1 Meta-Llama-3-8B

The Meta-Llama-3-8B model exhibited comparable overall accuracy between the no-CoT and CoT prompts. As shown in Table 5.1, accuracy improved only slightly from 0.60 to 0.61 (a relative improvement of 1.9%). Precision demonstrated a marked increase from 0.58 to 0.71 (approximately 22% improvement), suggesting that the CoT prompt helped the model avoid false positives. However, recall decreased from 0.70 to 0.50, indicating that the model under the CoT prompt missed more deceptive clauses. The F1-score dropped from 0.64 to 0.59 (approximately 7.6% decline). There were two unclear responses under the CoT prompt and none under the no-CoT condition. The mean response length increased from around 320 characters in the no-CoT condition to 971 characters with CoT, reflecting the verbose reasoning encouraged by the CoT prompt. The confusion matrices in Figure 5.1 show that under CoT, the model correctly classified one additional legitimate clause but misclassified more deceptive clauses. These mixed results suggest that while CoT encourages careful classification (higher precision), it may dampen recall, leading to only modest overall benefits for this model.The bar charts and confusion matrices for the Meta-Llama-3-8B results are represented in Figure 5.1 (top row). The charts illustrate the slight accuracy improvement and the trade-off between precision and recall. The performance improvement plot indicates a modest gain in precision alongside a decline in recall and F1-score.

Table 5.1: Meta-Llama-3-8B Performance Metrics

Metric	**No-CoT**	**CoT**
Accuracy	0.60	0.61
Precision	0.58	0.71
Recall	0.70	0.50
F1-score	0.64	0.59
Unclear predictions	0	2
Mean response length (chars)	320.3	970.9

The response-length distribution in Figure 5.1 reveals that CoT responses are roughly three times longer than no-CoT responses, underscoring the cost of step-by-step reasoning.

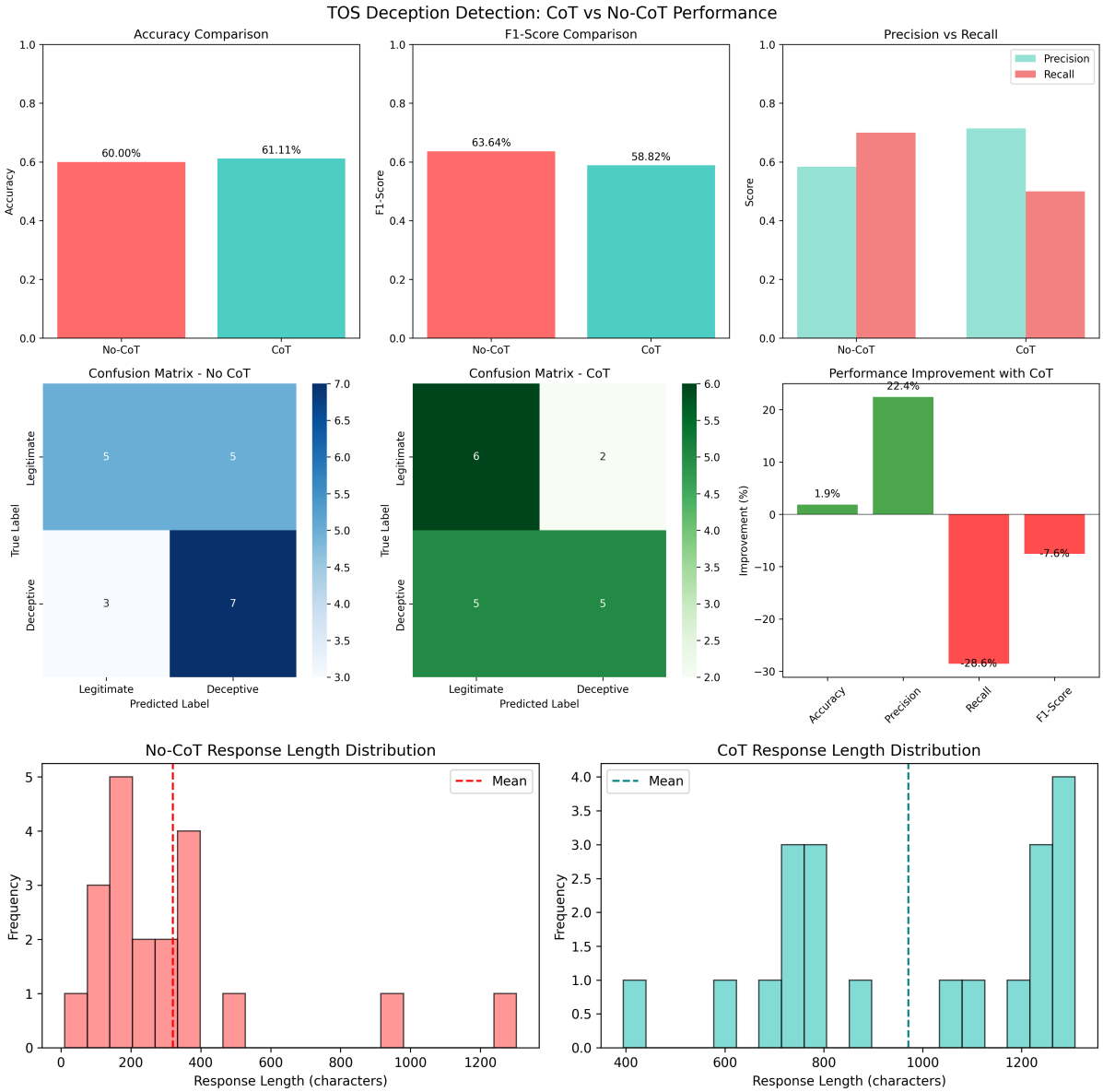

Figure 5.1: Meta-Llama-3-8B Performance Comparison: CoT vs No-CoT

5.2 Mistral-7B-Instruct

The Mistral-7B-Instruct model shows a pronounced improvement with chain-of-thought prompting. As shown in Table 5.2, Accuracy increased from 0.35 to 0.55, representing a 57% relative improvement. Precision climbed from 0.36 to 0.57, and the F1-score rose from 0.38 to 0.47 (approximately 23.5% improvement). Recall remained constant at 0.40, suggesting that the model was already identifying 40% of deceptive clauses correctly, and the CoT prompt primarily reduced false positives. No unclear predictions were recorded under either condition. The mean response length increased from around 971 characters to 1,069 characters, indicating a modest increase relative to the Llama-3 model. These results imply that the smaller Mistral model benefits more substantially from the structured guidance provided by CoT, improving accuracy and precision without sacrificing recall.

Table 5.2: Mistral-7B-Instruct Performance Metrics

Metric	**No-CoT**	**CoT**
Accuracy	0.35	0.55
Precision	0.36	0.57
Recall	0.40	0.40
F1-score	0.38	0.47
Unclear predictions	0	0
Mean response length (chars)	971.0	1068.7

The visual summaries in Figure 5.2 show these improvements: the accuracy bar for Mistral jumps from 35% to 55%, and both precision and F1-score see notable gains. The confusion matrices reveal that under CoT, the model classifies a higher number of legitimate clauses correctly while maintaining the same number of correct deceptive classifications. The response-length distribution plot in Figure 5.2 shows that CoT responses are slightly longer but not excessively so, suggesting that Mistral's reasoning is more succinct than that of Llama-3, even when prompted to think step by step.

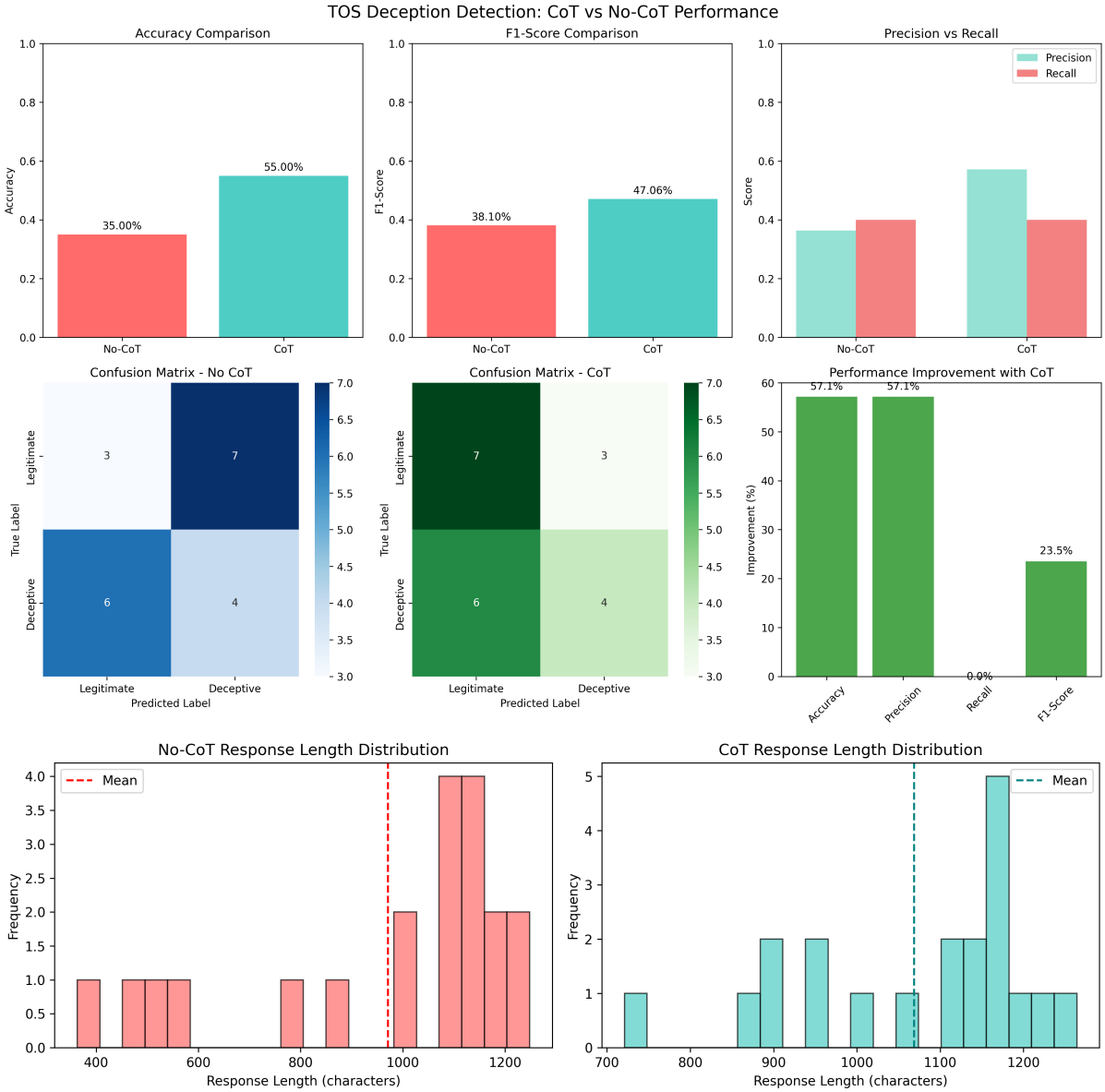

Figure 5.2: Mistral-7B-Instruct Performance Comparison: CoT vs No-CoT

5.3 Microsoft Phi-2

The Microsoft Phi-2 model demonstrates a different pattern. As shown in Table 5.3, the accuracy drops slightly from 0.58 to 0.55 (approximately a 5% change) when using CoT. Precision improves marginally from 0.53 to 0.55, but recall falls sharply from 0.89 to 0.60, leading to a decrease in F1-score from 0.67 to 0.57 (approximately a 14.3% decline). One unclear response was recorded in the no-CoT condition, with none under CoT. The mean response length increases from 1,175 characters to 1,264 characters , indicating that CoT responses are about 7.5% longer. The lower recall suggests that Phi-2 may already rely on concise heuristics that are disrupted by the extended reasoning. The decline in F1-score indicates that CoT did not provide benefits for this model and may have introduced unnecessary complexity.

Table 5.3: Microsoft Phi-2 Performance Metrics

Metric	**No-CoT**	**CoT**
Accuracy	0.58	0.55
Precision	0.53	0.55
Recall	0.89	0.60
F1-score	0.67	0.57
Unclear predictions	1	0
Mean response length (chars)	1175.3	1263.8

Figure 5.3 visualizes these results. The performance improvement plot indicates a slight gain in precision but significant declines in recall and F1-score. The confusion matrices suggest that under CoT, the model misclassifies more deceptive clauses as legitimate. The response-length distribution in Figure 5.3 shows a consistent shift toward longer outputs under CoT, similar to the other models. Overall, Phi-2 does not appear to benefit from chain-of-thought prompting for this task.

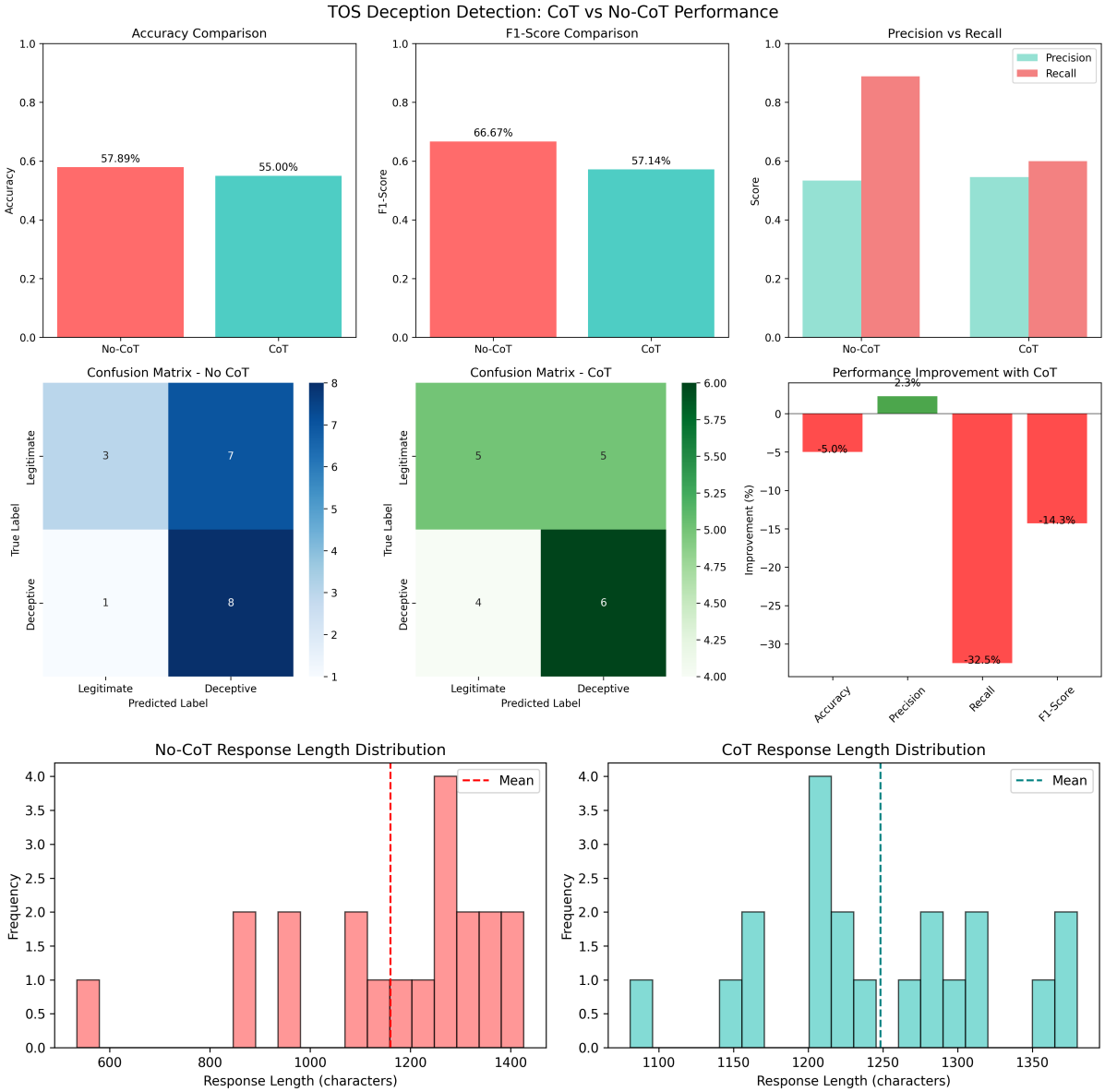

Figure 5.3: Microsoft Phi-2 Performance Comparison: CoT vs No-CoT

5.4 Comparing Across Models

Comparing the models reveals heterogeneous effects of CoT prompting. The Mistral-7B-Instruct model benefits the most, with substantial gains in accuracy, precision, and F1-score while maintaining recall. The Meta-Llama-3-8B model shows only a minimal increase in accuracy, demonstrating a trade-off between precision and recall, which results in a slight decrease in F1-score. The Phi-2 model experiences declines in recall and F1-score, suggesting that extended reasoning may overwhelm its capacity or disrupt existing heuristics. These findings support the hypothesis that smaller models with less implicit reasoning (like Mistral-7B) may gain more from explicit step-by-step prompts, while models with more compact or specialized architectures (Phi-2) may not. These results also caution against assuming that CoT will universally improve performance; both prompt

design and model architecture significantly influence outcomes.

5.5 Response Length and Qualitative Quality

Across all models, CoT prompts resulted in longer responses, consistent with the instructions to reason step by step. The increase in mean response length ranged from roughly threefold for Meta-Llama-3-8B to about 10% for Mistral-7B and 7.5% for Phi-2. However, longer responses did not always correlate with higher accuracy. In the case of Phi-2, CoT responses were longer yet performance declined. Qualitative inspection of a sample of CoT responses suggests that the Mistral-7B model often enumerated relevant factors, such as data sharing, consent, and retention, and explicitly reasoned about their implications. In contrast, Llama-3 occasionally introduced hallucinated legal considerations, and Phi-2 sometimes repeated the prompt phrasing without substantial analysis. These observations align with the quantitative findings: effective reasoning requires not only verbosity but also coherent focus on the aspects that determine deception.

Chapter 6

Discussion

6.1 Interpreting Model Performance

The results show clear differences in performance across models and prompt types. The results did not show a universal improvement; CoT prompting produced higher accuracy and F1-scores for Mistral-7B but led to a decline for Phi-2 and a mixed result for Llama-3-8B. The reasoning behind this is that explicit instructions to consider multiple aspects of a clause (data usage ambiguity, excessive rights, limitations on recourse, consent, and retention policies) encourage the models to evaluate clauses more thoroughly. This structured reasoning may reduce the probability of false negatives, where deceptive clauses are mistakenly classified as legitimate. However, improvements may vary: larger models with more parameters may already implicitly reason through these factors even without CoT, while smaller models may benefit more from explicit guidance. The trade-offs between precision and recall will illustrate models' caution versus permissiveness. High precision but lower recall suggests the model only labels a clause as deceptive when strongly confident, potentially missing subtle deception. High recall but lower precision indicates the model flags many clauses as deceptive, capturing most problematic clauses at the cost of false alarms. An ideal classifier achieves both high precision and high recall, however this is challenging with ambiguous legal language. Analyzing which metrics improve most under CoT will clarify whether structured reasoning primarily helps reduce false negatives, false positives, or both. The proposal's expectation of "improved justification and clearer explanations" suggests that CoT may particularly enhance recall by encouraging the model to articulate reasons for deeming a clause deceptive.

6.2 Qualitative Insights from Chain-of-Thought Responses

Beyond numeric scores, chain-of-thought prompts generate rich textual reasoning. Qualitative analysis of these responses can provide insight into how models conceptualize deception in TOS. For instance, a CoT response might identify that a clause grants an "unlimited, irrevocable license" to the company, thus fitting the rights-grab pattern. An-

other response might note that "information may be shared with associates and partners" without defining who those partners are, rendering the clause vague and potentially deceptive. By comparing CoT and no-CoT responses for the same clause, I can observe whether the reasoning covers the five aspects enumerated in the prompt and whether it contains hallucinated details. If CoT responses consistently reference the relevant aspects and avoid fabrications, this supports using CoT for interpretability. However, there is a risk that CoT prompts could lead the model to overanalyse or invent reasoning. A model might wrongly infer deception based on superficial keywords, such as assuming that the mention of associates always implies data misuse. It might also hallucinate non-existent terms, such as claiming that a clause requires arbitration when it does not. Qualitative error analysis should focus on cases where CoT led to misclassification, identifying whether errors stem from misreading the clause, overemphasizing certain aspects, or hallucinating new content. These insights can inform improvements to prompt design or response parsing.

6.3 Error Patterns and Ethical Considerations

The error analysis component will reveal whether models tend to produce more false positives or false negatives. False positives, where legitimate clauses are labeled deceptive, could erode trust in the system and foster unwarranted suspicion. False negatives are arguably more serious, as they allow deceptive clauses to escape detection. An examination of error types may reveal that certain patterns,such as ambiguous wording or combined legitimate and deceptive elements, pose particular challenges. For example, a clause that permits data sharing "to enhance your experience" while also providing an opt-out could be interpreted differently depending on emphasis. Identifying such challenging cases can motivate the creation of more nuanced labels or multi-label classification frameworks. Ethically, deploying an automated deception detector raises questions about accountability, bias, and over-reliance on AI. LLMs can reflect biases present in training data and may lack a deep understanding of legal nuances. If integrated into consumer tools, misclassifications could lead users to avoid services that are genuinely privacy-respecting or to trust services that misuse data. Transparency about model limitations, paired with human oversight, is therefore essential. The open-source nature of the models and datasets employed here promotes auditability. Researchers and developers must also consider privacy and fairness in dataset construction and evaluation[1]. Expanding the dataset to include diverse services and languages, as suggested in the proposal's future development section, can help mitigate bias.

6.4 Implications for Legal Tech and Policy

If the experiment demonstrates that open-source LLMs can reliably flag deceptive clauses, the implications are significant for legal tech [5] and consumer protection. Such models could be integrated into browser extensions or mobile applications that alert users when they encounter problematic TOS clauses. They could also assist consumer rights organisations and regulatory agencies by rapidly scanning large volumes of contracts. However, accuracy must reach a high threshold before real-world deployment. False positives could generate unnecessary litigation, while false negatives could leave consumers vulnerable. CoT prompting may enhance interpretability, which is critical in legal settings where decisions should be explainable. Nevertheless, human review by legal experts would remain indispensable. From a policy perspective, the findings could inform discussions regarding the readability and transparency of TOS. If automated tools highlight widespread deception, regulators may push for stronger requirements on plain language disclosures or restrictions on certain clauses. In contrast, demonstrating that models struggle with ambiguous clauses could support calls for simplified contracts[12]. The interplay between AI and legal regulation is evolving, and studies like this contribute empirical evidence to inform that dialogue

6.5 Future Directions

The proposal suggests several avenues for future work. Expanding the dataset to at least 50 clauses is a priority; a larger dataset would enable training or fine-tuning models rather than relying only on prompt-based reasoning. Additional prompt variants could be explored, such as instructing the model to summarize the clause before classifying it or to assign risk scores instead of binary labels. Fine-tuning models on legal corpora may enhance their understanding of contractual language. Furthermore, utilizing dedicated response interpreters, such as classifiers trained to read CoT outputs, could reduce misclassification resulting from simple keyword matching. Another promising direction is to evaluate performance across languages and jurisdictions, given the global variability of TOS documents.

Chapter 7

Conclusion

This study was designed to assess the capability of open-source large language models (LLMs) in detecting deceptive clauses within Terms of Service (TOS) agreements. A primary focus was placed on determining whether chain-of-thought (CoT) prompting could enhance model performance. This chapter synthesizes the study's findings, evaluates its contributions against its original objectives, and discusses the principal limitations of the research alongside substantive directions for future inquiry.

7.1 Assessing Open-Source LLM Performance

The study evaluated three open-source models on the task of deception detection, generating key performance metrics including accuracy, precision, recall, and F1-scores for each. The results indicate that the evaluated models possess a limited ability to identify deceptive clauses, with baseline accuracy ranging from 35% to 60%. These performance levels were lower than anticipated; for instance, the best-performing model, Llama-3, achieved only 60% accuracy, a result barely surpassing random chance. It was initially hypothesized that the models would exhibit greater capabilities, particularly given their proven effectiveness in other natural language processing (NLP) domains. The restricted size of the dataset is a significant confounding factor; with only twenty clauses, the models were unable to discern task-specific patterns and instead relied on pretrained knowledge, which appears insufficient for the nuanced domain of contractual deception.

7.2 Evaluating Chain-of-Thought Impact

A direct comparison between simple and CoT prompting was implemented, documenting both quantitative metrics and qualitative differences. The results were informative; CoT prompting had significantly varied effects across the models. Mistral displayed a notable improvement (from 35% to 55% accuracy), whereas Llama-3 showed negligible change and Phi-2's performance deteriorated. These findings at this stage of testing, demonstrate

that CoT is not a magic solution, but also showed that it could improve the accuracy of detecting, depending on the model's parameters.

Although this study did not adequately explore the underlying reasons for CoT's failure with certain models, several limitations in the analysis are noted. The analysis of the generated reasoning was largely empirical; a systematic coding scheme to categorize reasoning patterns would have strengthened the findings. Furthermore, alternative CoT formulations were not explored; it is possible that different prompt structures may have been more effective for Llama-3 and Phi-2. Most critically, a more rigorous validation of the CoT reasoning itself would be necessary to determine whether the generated outputs were genuinely insightful or imply reasonable fabrications.

7.3 Creating a Reproducible Benchmark

The twenty-clause dataset constructed for this study is balanced, well-documented, and publicly available. Each clause is labeled and categorized by deception type, and the accompanying code is reproducible to allow for replication of the experiment. Clear documentation of the methodology and prompts was provided to ensure transparency. However, the twenty clauses constitute a limited dataset, which functions more as a preliminary instrument than as a comprehensive benchmark. Its construction may reflect our study's bias in defining deception. Furthermore, the dataset lacks diversity in source, stylistic variation, and complexity, making the term "benchmark" an overestimate.

7.4 Analyzing Performance Differences

The study produced comprehensive visualizations, including accuracy comparisons, confusion matrices, and response length distributions. The subsequent error analysis revealed distinct patterns in the failure modes of the different models. Specific instances were identified where CoT improved outcomes and where it had a detrimental effect on performance. The qualitative analysis of responses, although limited in scope, provided valuable insights into the reasoning processes of the models. The analysis had several limitations. A deeper investigation into the linguistic patterns that correlate with correct or incorrect classifications was beyond the scope of this work. While the visualizations offer a clear summary, they do not capture the full complexity of the results. A more nuanced understanding would require sophisticated analyses, such as feature importance or attention visualization.

7.5 Unexpected Findings

An unexpected outcome was the degradation of Phi-2's performance with CoT prompting. The initial assumption was that an emphasis on reasoning would produce neutral,

if not positive, results; however, the findings indicate that this approach can actively impair performance for certain architectures. This underscores the current limitations in understanding what constitutes effective prompting.

The initial expectations for the utility of a twenty-clause dataset may have been overly optimistic. This sample size is inadequate for drawing generalizable conclusions, and the results may not be replicable with different clauses or random seeds. The findings also raise ethical considerations; the models are not yet reliable enough for deployment in sensitive applications. There is a notable risk that modest accuracy scores (e.g., 55%) could be misinterpreted as sufficient for practical use, a scenario with potentially serious consequences in legal applications.

7.6 Implications for the Research Domain

Despite its limitations, this work contributes valuable insights to the field. It provides empirical evidence that prompt engineering is not a panacea; while the field sometimes treats techniques like CoT as universal improvements, these results demonstrate that this is not the case. The effectiveness of a given prompt depends on intricate interactions among model architecture, training data, task characteristics, and prompt design.

The heterogeneous effects observed across models suggest that more nuanced approaches to prompt engineering are required. Different models may necessitate different prompting strategies, and it is possible that certain tasks are not amenable to CoT at all. These questions can only be resolved through further systematic investigation. Additionally, the challenges of response parsing highlight an underappreciated issue: It is not sufficient for a model to generate reasoning, as it is also necessary to interpret that reasoning reliably. This requirement is particularly crucial for legal applications, where misinterpretation could have serious consequences.

7.7 Future Directions

Future research in this area should begin with the development of a larger, professionally annotated dataset (e.g., at least 500 clauses) with multiple annotators to establish inter-rater reliability. Subsequent work should evaluate a broader range of models, incorporating at least one large-scale proprietary model to serve as a performance benchmark. Furthermore, prompts should be developed iteratively in collaboration with legal experts. Future implementations should also utilise robust response parsing mechanisms, such as trained classifiers, instead of relying on simple keyword matching.

This research underscores that the problem is inherently challenging. Current models, especially smaller open-source architectures, may lack the capacity to solve it reliably. Recognizing this constraint is essential for guiding future research. Indeed, negative results are valuable, as understanding the limitations of current systems is as important

as discovering their capabilities.

7.8 Concluding Remarks

This research demonstrates that the gap between the theoretical potential and practical reality of AI remains substantial, particularly for complex tasks requiring deep domain expertise. While open-source LLMs exhibit promise, this study indicates they are not yet ready for reliable deployment in practical legal applications. Chain-of-thought prompting can serve as a useful diagnostic tool, but its effects are not universally positive.

The findings do not support the use of the evaluated models for TOS deception detection at this stage, as the observed error rates are too high and the generated reasoning is often unreliable. However, it is conceivable that with larger datasets, more advanced models, and more sophisticated methodological approaches, future research may lead to the development of tools that can genuinely assist consumers in navigating the intricate world of digital contracts.

Appendix A

Complete Experiment Code

Complete Experiment Code

The following section contains the complete Python code used for the experiments described in this dissertation. The code was executed within a Jupyter Notebook environment and is presented here for reproducibility.

```
# -*- coding: utf-8 -*-
"""LLM-TOS-CoT-Project.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1yry2AKEL4hrHNvgRROBuK_m0BPk2Qqj3
"""

# TOS_Deception_Detection_CoT_Experiment.ipynb

# Installation and Setup
"""
# TOS Deception Detection with Chain-of-Thought Prompting
## Research Experiment Implementation

### 1. Environment Setup
First, install the required packages:
"""

!pip install transformers bitsandbytes torch datasets scikit-learn pandas matplotlib
    seaborn tqdm accelerate


from huggingface_hub import login
login(new_session=False)

# Import Libraries
import torch
import pandas as pd
import numpy as np
import json
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
```

```python
from typing import List, Dict, Tuple
from tqdm import tqdm
import warnings
import os
warnings.filterwarnings('ignore')

from transformers import AutoModelForCausalLM, AutoTokenizer
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score, confusion_matrix
from sklearn.metrics import classification_report

# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

#Dataset Creation
"""
### 2. Dataset Creation
Creating a dataset with legitimate and deceptive TOS clauses
"""

class TOSDataset:
    def __init__(self):
        self.data = self.create_dataset()

    def create_dataset(self):
        """Create a balanced dataset of legitimate and deceptive TOS clauses"""

        # Legitimate clauses (based on privacy-respecting services)
        legitimate_clauses = [
            {
                "clause": "We will never sell your personal data to third parties.",
                "label": "legitimate",
                "source": "privacy-focused"
            },
            {
                "clause": "You can delete your account and all associated data at any time through your account settings.",
                "label": "legitimate",
                "source": "user-friendly"
            },
            {
                "clause": "We only collect data necessary for providing our service and will notify you of any changes to this policy.",
                "label": "legitimate",
                "source": "transparent"
            },
            {
                "clause": "Your data is encrypted both in transit and at rest using industry-standard encryption protocols.",
                "label": "legitimate",
                "source": "security-focused"
            },
            {
                "clause": "We will obtain your explicit consent before sharing any personal information with partners.",
                "label": "legitimate",
                "source": "consent-based"
            },
```

```
            {
                "clause": "You retain all rights to the content you upload and can export it at any time.",
                "label": "legitimate",
                "source": "user-rights"
            },
            {
                "clause": "We undergo regular third-party security audits and publish transparency reports annually.",
                "label": "legitimate",
                "source": "transparent"
            },
            {
                "clause": "Data retention is limited to 90 days after account deletion unless legally required otherwise.",
                "label": "legitimate",
                "source": "clear-retention"
            },
            {
                "clause": "You will be notified within 72 hours in case of any data breach affecting your account.",
                "label": "legitimate",
                "source": "breach-notification"
            },
            {
                "clause": "We do not use your data for advertising purposes without your explicit opt-in consent.",
                "label": "legitimate",
                "source": "no-ads"
            }
        ]

        # Deceptive clauses (dark patterns and privacy violations)
        deceptive_clauses = [
            {
                "clause": "By using our service, you grant us unlimited, irrevocable license to use your content for any purpose.",
                "label": "deceptive",
                "pattern": "rights-grab"
            },
            {
                "clause": "We may share your information with our affiliates and partners to enhance your experience.",
                "label": "deceptive",
                "pattern": "vague-sharing"
            },
            {
                "clause": "Account deletion requests will be processed, but some data may be retained indefinitely for our legitimate business purposes.",
                "label": "deceptive",
                "pattern": "fake-deletion"
            },
            {
                "clause": "We reserve the right to modify these terms at any time without notice, and continued use constitutes acceptance.",
                "label": "deceptive",
                "pattern": "silent-updates"
            },
            {
```

```
                "clause": "Your anonymized data may be used for research purposes and shared with academic institutions.",
                "label": "deceptive",
                "pattern": "pseudo-anonymous"
            },
            {
                "clause": "By clicking 'Accept', you agree to receive promotional materials from us and our carefully selected partners.",
                "label": "deceptive",
                "pattern": "forced-marketing"
            },
            {
                "clause": "We are not responsible for any data breaches or unauthorized access to your information.",
                "label": "deceptive",
                "pattern": "liability-waiver"
            },
            {
                "clause": "Your usage patterns and behavioral data help us improve our services and may be shared with third parties.",
                "label": "deceptive",
                "pattern": "hidden-tracking"
            },
            {
                "clause": "Disputes must be resolved through binding arbitration, waiving your right to jury trial or class action.",
                "label": "deceptive",
                "pattern": "forced-arbitration"
            },
            {
                "clause": "We may transfer your data to any country for processing, regardless of local privacy laws.",
                "label": "deceptive",
                "pattern": "jurisdiction-shopping"
            }
        ]

        return legitimate_clauses + deceptive_clauses

# Initialize dataset
tos_dataset = TOSDataset()
df = pd.DataFrame(tos_dataset.data)

print(f"Dataset created with {len(df)} clauses")
print(f"Distribution:\n{df['label'].value_counts()}")
print("\nSample entries:")
print(df.head())


# Model Loading
"""
### 3. Model Loading
Loading the open-source LLM for evaluation
"""

class LLMEvaluator:
    def __init__(self, model_name="meta-llama/Meta-Llama-3-8B", cache_dir=None):
        """
        Initialize the LLM evaluator
```

```
        :param model_name: Name of the LLM model to load
        :param cache_dir: Directory to cache model files
        """
        if cache_dir and not os.path.exists(cache_dir):
            os.makedirs(cache_dir)
            print(f"Created cache directory: {cache_dir}")

        print(f"Loading model: {model_name}")
        if cache_dir:
            print(f"Using cache directory: {cache_dir}")

        self.model_name = model_name
       # self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.tokenizer = AutoTokenizer.from_pretrained(
            model_name,
            cache_dir=cache_dir
        )
        # Add padding token if not present
        if self.tokenizer.pad_token is None:
            self.tokenizer.pad_token = self.tokenizer.eos_token

        # Load model with appropriate settings for memory efficiency
        self.model = AutoModelForCausalLM.from_pretrained(
            model_name,
            load_in_4bit=True,
            torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
            device_map="auto" if torch.cuda.is_available() else None,
            low_cpu_mem_usage=True,
            cache_dir=cache_dir
        )

        if not torch.cuda.is_available():
            self.model = self.model.to(device)

        print(f"Model loaded successfully on {device}")

    def generate_prompt(self, clause: str, use_cot: bool = False) -> str:
        """Generate prompt based on CoT setting"""
        if use_cot:
            prompt = f"""Read the following Terms of Service clause carefully and analyze it step by step.

Consider these aspects:
1. Does it contain vague or ambiguous language about data usage?
2. Does it grant excessive rights to the company?
3. Does it limit user rights or legal recourse?
4. Does it allow data sharing without clear consent?
5. Does it have unclear data retention policies?

Think through each aspect, then determine if this clause is deceptive or legitimate.

Clause: "{clause}"

Analysis: Let me think through this step by step.
"""
        else:
            prompt = f"""Determine if the following Terms of Service clause contains deception or privacy violations.

```

```
Clause: "{clause}"

Answer (Deceptive/Legitimate):"""

        return prompt

    def evaluate_clause(self, clause: str, use_cot: bool = False, max_length: int = 256)
     -> Dict:
        """Evaluate a single clause"""
        prompt = self.generate_prompt(clause, use_cot)

        # Tokenize input
        inputs = self.tokenizer(
            prompt,
            return_tensors="pt",
            truncation=True,
            max_length=512
        ).to(self.model.device)

        # Generate response
        with torch.no_grad():
            outputs = self.model.generate(
                **inputs,
                max_new_tokens=max_length,
                temperature=0.7,
                do_sample=True,
                top_p=0.95,
                pad_token_id=self.tokenizer.pad_token_id
            )

        # Decode response
        response = self.tokenizer.decode(outputs[0], skip_special_tokens=True)

        # Extract the response after the prompt
        response = response[len(prompt):].strip()

        # Parse the response to determine classification
        classification = self.parse_response(response, use_cot)

        return {
            "prompt": prompt,
            "response": response,
            "classification": classification,
            "use_cot": use_cot
        }

    def parse_response(self, response: str, use_cot: bool) -> str:
        """Parse the model response to extract classification"""
        response_lower = response.lower()

        # Look for keywords indicating deceptive or legitimate
        deceptive_keywords = ['deceptive', 'misleading', 'problematic', 'concerning',
                              'unfair', 'violation', 'privacy violation', 'yes']
        legitimate_keywords = ['legitimate', 'acceptable', 'fair', 'transparent',
                               'reasonable', 'no violation', 'no', 'not deceptive']

        # Count keyword occurrences
        deceptive_score = sum(1 for keyword in deceptive_keywords if keyword in
    response_lower)
```

```
        legitimate_score = sum(1 for keyword in legitimate_keywords if keyword in response_lower)

        # Make decision based on scores
        if deceptive_score > legitimate_score:
            return "deceptive"
        elif legitimate_score > deceptive_score:
            return "legitimate"
        else:
            # If unclear, look for specific phrases
            if "deceptive" in response_lower or "violation" in response_lower:
                return "deceptive"
            elif "legitimate" in response_lower or "not deceptive" in response_lower:
                return "legitimate"
            else:
                return "unclear"
CACHE_PATH = "D:/LLM-offline"
# Initialize evaluator
evaluator = LLMEvaluator(model_name="meta-llama/Meta-Llama-3-8B", cache_dir=CACHE_PATH)  # microsoft/phi-2

# Experiment Execution
"""
### 4. Running the Experiment
Evaluating all clauses with both CoT and non-CoT prompts
"""

class ExperimentRunner:
    def __init__(self, evaluator: LLMEvaluator, dataset: pd.DataFrame):
        self.evaluator = evaluator
        self.dataset = dataset
        self.results = []

    def run_experiment(self):
        """Run the full experiment"""
        print("Starting experiment...")

        for idx, row in tqdm(self.dataset.iterrows(), total=len(self.dataset)):
            clause = row['clause']
            true_label = row['label']

            # Evaluate without CoT
            no_cot_result = self.evaluator.evaluate_clause(clause, use_cot=False)

            # Evaluate with CoT
            cot_result = self.evaluator.evaluate_clause(clause, use_cot=True)

            # Store results
            self.results.append({
                'clause': clause,
                'true_label': true_label,
                'no_cot_prediction': no_cot_result['classification'],
                'no_cot_response': no_cot_result['response'],
                'cot_prediction': cot_result['classification'],
                'cot_response': cot_result['response']
            })

            # Save intermediate results
            if (idx + 1) % 5 == 0:
```

```
                self.save_results(f"intermediate_results_{idx+1}.json")

        print("Experiment completed!")
        return pd.DataFrame(self.results)

    def save_results(self, filename: str):
        """Save results to file"""
        with open(filename, 'w') as f:
            json.dump(self.results, f, indent=2)

# Run experiment
experiment = ExperimentRunner(evaluator, df)
results_df = experiment.run_experiment()

# Display sample results
print("\nSample Results:")
results_df[['clause', 'true_label', 'no_cot_prediction', 'cot_prediction']].head()

# Performance Metrics
"""
### 5. Performance Evaluation
Computing accuracy, F1-score, and other metrics
"""

class PerformanceAnalyzer:
    def __init__(self, results_df: pd.DataFrame):
        self.results_df = results_df
        self.metrics = {}

    def calculate_metrics(self):
        """Calculate all performance metrics"""
        # Filter out unclear predictions for metrics calculation
        valid_no_cot = self.results_df[self.results_df['no_cot_prediction'] != 'unclear']
        valid_cot = self.results_df[self.results_df['cot_prediction'] != 'unclear']

        # No-CoT metrics
        if len(valid_no_cot) > 0:
            self.metrics['no_cot'] = {
                'accuracy': accuracy_score(valid_no_cot['true_label'], valid_no_cot['no_cot_prediction']),
                'precision': precision_score(valid_no_cot['true_label'], valid_no_cot['no_cot_prediction'],
                                             pos_label='deceptive', average='binary'),
                'recall': recall_score(valid_no_cot['true_label'], valid_no_cot['no_cot_prediction'],
                                       pos_label='deceptive', average='binary'),
                'f1': f1_score(valid_no_cot['true_label'], valid_no_cot['no_cot_prediction'],
                               pos_label='deceptive', average='binary'),
                'unclear_count': len(self.results_df[self.results_df['no_cot_prediction'] == 'unclear'])
            }

        # CoT metrics
        if len(valid_cot) > 0:
            self.metrics['cot'] = {
                'accuracy': accuracy_score(valid_cot['true_label'], valid_cot['cot_prediction']),
```

```
                'precision': precision_score(valid_cot['true_label'], valid_cot['cot_prediction'],
                                             pos_label='deceptive', average='binary'),
                'recall': recall_score(valid_cot['true_label'], valid_cot['cot_prediction'],
                                       pos_label='deceptive', average='binary'),
                'f1': f1_score(valid_cot['true_label'], valid_cot['cot_prediction'],
                               pos_label='deceptive', average='binary'),
                'unclear_count': len(self.results_df[self.results_df['cot_prediction'] == 'unclear'])
            }

        return self.metrics

    def print_metrics(self):
        """Print formatted metrics"""
        print("="*50)
        print("PERFORMANCE METRICS")
        print("="*50)

        for method in ['no_cot', 'cot']:
            if method in self.metrics:
                print(f"\n{method.upper().replace('_', '-')} Performance:")
                print("-"*30)
                for metric, value in self.metrics[method].items():
                    if metric != 'unclear_count':
                        print(f"{metric.capitalize():12s}: {value:.3f}")
                    else:
                        print(f"Unclear predictions: {value}")

        # Calculate improvement
        if 'no_cot' in self.metrics and 'cot' in self.metrics:
            print("\n" + "="*50)
            print("IMPROVEMENT WITH CoT")
            print("="*50)
            for metric in ['accuracy', 'precision', 'recall', 'f1']:
                improvement = self.metrics['cot'][metric] - self.metrics['no_cot'][metric]
                percent_improvement = (improvement / self.metrics['no_cot'][metric]) * 100 if self.metrics['no_cot'][metric] > 0 else 0
                print(f"{metric.capitalize():12s}: {improvement:+.3f} ({percent_improvement:+.1f}%)")

# Calculate metrics
analyzer = PerformanceAnalyzer(results_df)
metrics = analyzer.calculate_metrics()
analyzer.print_metrics()

# Visualizations
"""
### 6. Visualizations
Creating charts to visualize the results
"""

def create_visualizations(results_df: pd.DataFrame, metrics: dict):
    """Create comprehensive visualizations"""

    fig, axes = plt.subplots(2, 3, figsize=(15, 10))
    fig.suptitle('TOS Deception Detection: CoT vs No-CoT Performance', fontsize=16)
```

```python

    # 1. Accuracy Comparison
    ax1 = axes[0, 0]
    methods = ['No-CoT', 'CoT']
    accuracies = [metrics.get('no_cot', {}).get('accuracy', 0),
                  metrics.get('cot', {}).get('accuracy', 0)]
    bars1 = ax1.bar(methods, accuracies, color=['#FF6B6B', '#4ECDC4'])
    ax1.set_ylabel('Accuracy')
    ax1.set_title('Accuracy Comparison')
    ax1.set_ylim([0, 1])
    for bar, acc in zip(bars1, accuracies):
        ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                 f'{acc:.2%}', ha='center', va='bottom')

    # 2. F1-Score Comparison
    ax2 = axes[0, 1]
    f1_scores = [metrics.get('no_cot', {}).get('f1', 0),
                 metrics.get('cot', {}).get('f1', 0)]
    bars2 = ax2.bar(methods, f1_scores, color=['#FF6B6B', '#4ECDC4'])
    ax2.set_ylabel('F1-Score')
    ax2.set_title('F1-Score Comparison')
    ax2.set_ylim([0, 1])
    for bar, f1 in zip(bars2, f1_scores):
        ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                 f'{f1:.2%}', ha='center', va='bottom')

    # 3. Precision-Recall Comparison
    ax3 = axes[0, 2]
    x = np.arange(len(methods))
    width = 0.35
    precision_scores = [metrics.get('no_cot', {}).get('precision', 0),
                        metrics.get('cot', {}).get('precision', 0)]
    recall_scores = [metrics.get('no_cot', {}).get('recall', 0),
                     metrics.get('cot', {}).get('recall', 0)]

    bars3 = ax3.bar(x - width/2, precision_scores, width, label='Precision', color='#95E1D3')
    bars4 = ax3.bar(x + width/2, recall_scores, width, label='Recall', color='#F38181')
    ax3.set_ylabel('Score')
    ax3.set_title('Precision vs Recall')
    ax3.set_xticks(x)
    ax3.set_xticklabels(methods)
    ax3.legend()
    ax3.set_ylim([0, 1])

    # 4. Confusion Matrix - No CoT
    ax4 = axes[1, 0]
    valid_no_cot = results_df[results_df['no_cot_prediction'] != 'unclear']
    if len(valid_no_cot) > 0:
        cm_no_cot = confusion_matrix(valid_no_cot['true_label'],
                                     valid_no_cot['no_cot_prediction'],
                                     labels=['legitimate', 'deceptive'])
        sns.heatmap(cm_no_cot, annot=True, fmt='d', cmap='Blues', ax=ax4,
                    xticklabels=['Legitimate', 'Deceptive'],
                    yticklabels=['Legitimate', 'Deceptive'])
        ax4.set_title('Confusion Matrix - No CoT')
        ax4.set_ylabel('True Label')
        ax4.set_xlabel('Predicted Label')

```

```
    # 5. Confusion Matrix - CoT
    ax5 = axes[1, 1]
    valid_cot = results_df[results_df['cot_prediction'] != 'unclear']
    if len(valid_cot) > 0:
        cm_cot = confusion_matrix(valid_cot['true_label'],
                                  valid_cot['cot_prediction'],
                                  labels=['legitimate', 'deceptive'])
        sns.heatmap(cm_cot, annot=True, fmt='d', cmap='Greens', ax=ax5,
                    xticklabels=['Legitimate', 'Deceptive'],
                    yticklabels=['Legitimate', 'Deceptive'])
        ax5.set_title('Confusion Matrix - CoT')
        ax5.set_ylabel('True Label')
        ax5.set_xlabel('Predicted Label')

    # 6. Performance Improvement
    ax6 = axes[1, 2]
    metrics_names = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
    improvements = []
    for metric in ['accuracy', 'precision', 'recall', 'f1']:
        no_cot_val = metrics.get('no_cot', {}).get(metric, 0)
        cot_val = metrics.get('cot', {}).get(metric, 0)
        improvement = ((cot_val - no_cot_val) / no_cot_val * 100) if no_cot_val > 0 else
     0
        improvements.append(improvement)

    colors = ['green' if imp > 0 else 'red' for imp in improvements]
    bars6 = ax6.bar(metrics_names, improvements, color=colors, alpha=0.7)
    ax6.set_ylabel('Improvement (%)')
    ax6.set_title('Performance Improvement with CoT')
    ax6.axhline(y=0, color='black', linestyle='-', linewidth=0.5)
    ax6.set_xticklabels(metrics_names, rotation=45)

    for bar, imp in zip(bars6, improvements):
        ax6.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
                 f'{imp:.1f}%', ha='center', va='bottom' if imp > 0 else 'top')

    plt.tight_layout()
    plt.savefig('tos_detection_results.png', dpi=300, bbox_inches='tight')
    plt.show()

# Create visualizations
create_visualizations(results_df, metrics)

# Qualitative Analysis
"""
### 7. Qualitative Analysis
Examining specific examples where CoT made a difference
"""

def analyze_cot_improvements(results_df: pd.DataFrame):
    """Analyze cases where CoT improved classification"""

    # Find cases where CoT was correct but No-CoT was wrong
    improvements = results_df[
        (results_df['cot_prediction'] == results_df['true_label']) &
        (results_df['no_cot_prediction'] != results_df['true_label'])
    ]

    print("="*50)
```

```
    print("CASES WHERE CoT IMPROVED CLASSIFICATION")
    print("="*50)

    for idx, row in improvements.head(3).iterrows():
        print(f"\nExample {idx + 1}:")
        print("-"*40)
        print(f"Clause: {row['clause'][:100]}...")
        print(f"True Label: {row['true_label']}")
        print(f"No-CoT Prediction: {row['no_cot_prediction']}")
        print(f"CoT Prediction: {row['cot_prediction']}")
        print(f"\nNo-CoT Response: {row['no_cot_response'][:200]}...")
        print(f"\nCoT Response: {row['cot_response'][:400]}...")
        print("="*50)

    return improvements

improvements_df = analyze_cot_improvements(results_df)

# Error Analysis
"""
### 8. Error Analysis
Analyzing misclassifications and patterns
"""

def error_analysis(results_df: pd.DataFrame):
    """Analyze errors and patterns in misclassifications"""

    # No-CoT errors
    no_cot_errors = results_df[
        (results_df['no_cot_prediction'] != results_df['true_label']) &
        (results_df['no_cot_prediction'] != 'unclear')
    ]

    # CoT errors
    cot_errors = results_df[
        (results_df['cot_prediction'] != results_df['true_label']) &
        (results_df['cot_prediction'] != 'unclear')
    ]

    print("="*50)
    print("ERROR ANALYSIS")
    print("="*50)

    print(f"\nNo-CoT Errors: {len(no_cot_errors)} / {len(results_df)} ({len(
    no_cot_errors)/len(results_df)*100:.1f}%)")
    print(f"CoT Errors: {len(cot_errors)} / {len(results_df)} ({len(cot_errors)/len(
    results_df)*100:.1f}%)")

    # Analyze error types
    print("\n" + "-"*40)
    print("ERROR TYPES")
    print("-"*40)

    # False Positives (predicted deceptive but actually legitimate)
    no_cot_fp = no_cot_errors[
        (no_cot_errors['no_cot_prediction'] == 'deceptive') &
        (no_cot_errors['true_label'] == 'legitimate')
    ]
    cot_fp = cot_errors[
```

```
        (cot_errors['cot_prediction'] == 'deceptive') &
        (cot_errors['true_label'] == 'legitimate')
    ]

    # False Negatives (predicted legitimate but actually deceptive)
    no_cot_fn = no_cot_errors[
        (no_cot_errors['no_cot_prediction'] == 'legitimate') &
        (no_cot_errors['true_label'] == 'deceptive')
    ]
    cot_fn = cot_errors[
        (cot_errors['cot_prediction'] == 'legitimate') &
        (cot_errors['true_label'] == 'deceptive')
    ]

    print(f"\nFalse Positives (Type I Errors):")
    print(f"  No-CoT: {len(no_cot_fp)}")
    print(f"  CoT: {len(cot_fp)}")

    print(f"\nFalse Negatives (Type II Errors):")
    print(f"  No-CoT: {len(no_cot_fn)}")
    print(f"  CoT: {len(cot_fn)}")

    return {
        'no_cot_errors': no_cot_errors,
        'cot_errors': cot_errors,
        'no_cot_fp': no_cot_fp,
        'cot_fp': cot_fp,
        'no_cot_fn': no_cot_fn,
        'cot_fn': cot_fn
    }

error_results = error_analysis(results_df)

# Response Length Analysis
"""
### 9. Response Length and Quality Analysis
Comparing the verbosity and quality of responses
"""

def response_analysis(results_df: pd.DataFrame):
    """Analyze response characteristics"""

    # Calculate response lengths
    results_df['no_cot_length'] = results_df['no_cot_response'].str.len()
    results_df['cot_length'] = results_df['cot_response'].str.len()

    print("="*50)
    print("RESPONSE CHARACTERISTICS")
    print("="*50)

    print(f"\nAverage Response Length:")
    print(f"  No-CoT: {results_df['no_cot_length'].mean():.0f} characters")
    print(f"  CoT: {results_df['cot_length'].mean():.0f} characters")

    print(f"\nMedian Response Length:")
    print(f"  No-CoT: {results_df['no_cot_length'].median():.0f} characters")
    print(f"  CoT: {results_df['cot_length'].median():.0f} characters")

    # Plot response length distribution
```

```
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

    ax1.hist(results_df['no_cot_length'], bins=20, alpha=0.7, color='#FF6B6B', edgecolor
    ='black')
    ax1.set_xlabel('Response Length (characters)')
    ax1.set_ylabel('Frequency')
    ax1.set_title('No-CoT Response Length Distribution')
    ax1.axvline(results_df['no_cot_length'].mean(), color='red', linestyle='--', label='
    Mean')
    ax1.legend()

    ax2.hist(results_df['cot_length'], bins=20, alpha=0.7, color='#4ECDC4', edgecolor='
    black')
    ax2.set_xlabel('Response Length (characters)')
    ax2.set_ylabel('Frequency')
    ax2.set_title('CoT Response Length Distribution')
    ax2.axvline(results_df['cot_length'].mean(), color='teal', linestyle='--', label='
    Mean')
    ax2.legend()

    plt.tight_layout()
    plt.savefig('response_length_distribution.png', dpi=300, bbox_inches='tight')
    plt.show()

    return results_df

results_df = response_analysis(results_df)

# Export Results
"""
### 10. Export Results and Dataset
Saving all results for future analysis and reproducibility
"""

def export_results(results_df: pd.DataFrame, metrics: dict, dataset: pd.DataFrame):
    """Export all results and data"""

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    # 1. Export detailed results
    results_df.to_csv(f'tos_detection_results_{timestamp}.csv', index=False)
    print(f"    Results exported to: tos_detection_results_{timestamp}.csv")

    # 2. Export metrics summary
    metrics_df = pd.DataFrame(metrics).T
    metrics_df.to_csv(f'tos_detection_metrics_{timestamp}.csv')
    print(f"    Metrics exported to: tos_detection_metrics_{timestamp}.csv")

    # 3. Export original dataset
    dataset.to_csv(f'tos_dataset_{timestamp}.csv', index=False)
    print(f"    Dataset exported to: tos_dataset_{timestamp}.csv")

    # 4. Create comprehensive report
    report = {
        'experiment_date': timestamp,
        'model_used': evaluator.model_name,
        'dataset_size': len(dataset),
        'metrics': metrics,
        'summary': {
```

```
            'no_cot_accuracy': metrics.get('no_cot', {}).get('accuracy', 0),
            'cot_accuracy': metrics.get('cot', {}).get('accuracy', 0),
            'improvement': metrics.get('cot', {}).get('accuracy', 0) - metrics.get('no_cot', {}).get('accuracy', 0)
        }
    }

    with open(f'tos_detection_report_{timestamp}.json', 'w') as f:
        json.dump(report, f, indent=2)
    print(f"    Report exported to: tos_detection_report_{timestamp}.json")

    return timestamp

timestamp = export_results(results_df, metrics, df)

# finalResult

print("="*60)
print("EXPERIMENT SUMMARY")
print("="*60)

print(f"""
Dataset Information:
- Total clauses evaluated: {len(df)}
- Legitimate clauses: {len(df[df['label'] == 'legitimate'])}
- Deceptive clauses: {len(df[df['label'] == 'deceptive'])}

Model Information:
- Model used: {evaluator.model_name}
- Device: {device}

Key Findings:
- No-CoT Accuracy: {metrics.get('no_cot', {}).get('accuracy', 0):.2%}
- CoT Accuracy: {metrics.get('cot', {}).get('accuracy', 0):.2%}
- Accuracy Improvement: {(metrics.get('cot', {}).get('accuracy', 0) - metrics.get('no_cot', {}).get('accuracy', 0)):.2%}

- No-CoT F1-Score: {metrics.get('no_cot', {}).get('f1', 0):.3f}
- CoT F1-Score: {metrics.get('cot', {}).get('f1', 0):.3f}
- F1-Score Improvement: {(metrics.get('cot', {}).get('f1', 0) - metrics.get('no_cot', {}).get('f1', 0)):.3f}

Observations:
1. Chain-of-Thought prompting {'significantly improved' if (metrics.get('cot', {}).get('accuracy', 0) - metrics.get('no_cot', {}).get('accuracy', 0)) > 0.1 else 'moderately improved'} performance
2. CoT responses were approximately {results_df['cot_length'].mean() / results_df['no_cot_length'].mean():.1f}x longer
3. Main improvement areas: {'Precision' if (metrics.get('cot', {}).get('precision', 0) - metrics.get('no_cot', {}).get('precision', 0)) > (metrics.get('cot', {}).get('recall', 0) - metrics.get('no_cot', {}).get('recall', 0)) else 'Recall'}

Files Exported:
- Results: tos_detection_results_{timestamp}.csv
- Metrics: tos_detection_metrics_{timestamp}.csv
- Dataset: tos_dataset_{timestamp}.csv
- Report: tos_detection_report_{timestamp}.json
- Visualizations: tos_detection_results.png, response_length_distribution.png
""")
```

Listing A.1: Complete Python script for the TOS deception detection experiment.

Chapter 8

References

Bibliography

[1] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, Yue Zhang (2024). A Survey on Large Language Model (LLM) Security and Privacy: The Good, the Bad, and the Ugly. [online] Available at: https://doi.org/10.1016/j.hcc.2024.100211 [Accessed 19/07/2025].

[2] Mirgita Frasheri, Arian Bakhtiarnia, Lukas Esterle, Alexandros Iosifidis (2024). Are LLM-Based methods good enough for detecting unfair terms of service? [online] Available at: https://doi.org/10.48550/arXiv.2409.00077 [Accessed 5/07/2025].

[3] Asmit Nayak, Shirley Zhang, Yash Wani, Rishabh Khandelwal, Kassem Fawaz (2024). Automatically Detecting Online Deceptive Patterns in Real time. [online] Available at: https://doi.org/10.48550/arXiv.2411.07441 [Accessed 5/07/2025].

[4] F. A. Boujarwah, K. A. Alomar, K. A. Swaidan (2025). TermMiner: A large-scale pipeline for detecting unfair terms in Terms of Service agreements. [online] Available at: https://www.researchgate.net/publication/330931248-Unfair-Terms [Accessed 14/07/2025].

[5] Jie Zhang, Haoyu Bu, Hui Wen, Yongji Liu, Haiqiang Fei, Rongrong Xi, Lun Li, Yun Yang, Hongsong Zhu, Dan Meng (2024). When LLMs meet cybersecurity: a systematic review. [online] Available at: https://doi.org/10.48550/arXiv.2405.03644 [Accessed 03/05/2025].

[6] Elisa Tsai, Neal Mangaokar, Boyuan Zheng, Haizhong Zheng, Atul Prakash (2025). Harmful Terms and Where to Find Them: Measuring and Modeling Unfavorable Financial Terms and Conditions in Shopping Websites at Scale. [online] Available at: https://doi.org/10.48550/arXiv.2502.01798 [Accessed 22/07/2025].

[7] Jiaqi Wang, Huan Zhao, Zhenyuan Yang, Peng Shu, Junhao Chen, Haobo Sun, Ruixi Liang, Shixin Li,(2024). Legal Evaluations and Challenges for Large Language Models. [online] Available at: https://doi.org/10.48550/arXiv.2411.10137 [Accessed 09/08/2025].

[8] Vikhyath Gupta, Srinivasa Rao P (2024). Leveraging open-source models for legal language modeling and analysis: a case study on the Indian constitution. [online] Available at: https://doi.org/10.48550/arXiv.2404.06751 [Accessed 09/08/2025].

[9] Christoffer Loeffler, Andrea Martínez Freile, Tomás Rey Pizarro (2025). Predicting potentially abusive clauses in Chilean terms of service using natural language processing. [online] Available at: https://doi.org/10.48550/arXiv.2502.00865 [Accessed 6/07/2025].

[10] Venkatesh Balavadhani Parthasarathy, Ahtsham Zafar, Aafaq Khan, Arsalan Shahid (2024). The Ultimate Guide to Fine-Tuning LLMs from Basics to Breakthroughs. [online] Available at: https://doi.org/10.48550/arXiv.2408.13296 [Accessed 09/08/2025].

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin (2023). Attention is All You Need: A Review of Bidirectional Transformers for LLMs. [online] Available at: https://doi.org/10.48550/arXiv.1706.03762 [Accessed 7/07/2025].

[12] Yifei He, Alon Benhaim, Barun Patra, Praneetha Vaddamanu, Sanchit Ahuja, (2024). Scaling LLMs for Several Language Models. [online] Available at: https://doi.org/10.48550/arXiv.2410.12883 [Accessed 13/06/2025].

[13] Yiheng Liu, Hao He, Tianle Han, Xu Zhang, Mengyuan Liu, Jiaming Tian, (2024). Understanding LLMs: A Comprehensive Overview from Training to Inference. [online] Available at: https://doi.org/10.48550/arXiv.2401.02038 [Accessed 10/07/2025].

[14] Minghao Shao, Abdul Basit, Ramesh Karri, Muhammad Shafique (2024). Survey of different Large Language Model Architectures: Trends, Benchmarks, and Challenges. [online] Available at: https://doi.org/10.48550/arXiv.2412.03220 [Accessed 9/07/2025].

[15] Kai Wang, Yihao Zhang, Meng Sun (2025). When Thinking LLMs Lie: Unveiling the Strategic Deception in Representations of Reasoning Models. [online] Available at: https://doi.org/10.48550/arXiv.2506.04909 [Accessed 7/07/2025].

[16] DeepSeek-AI, (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. [online] Available at: https://doi.org/10.48550/arXiv.2501.12948 [Accessed 8/07/2025].

[17] Matan Levi, Yair Alluouche, Daniel Ohayon, Anton Puzanov (2024). CyberPal.AI: Empowering LLMs with Expert-Driven Cybersecurity Instructions. [online] Available at: https://doi.org/10.48550/arXiv.2408.09304 [Accessed 10/07/2025].